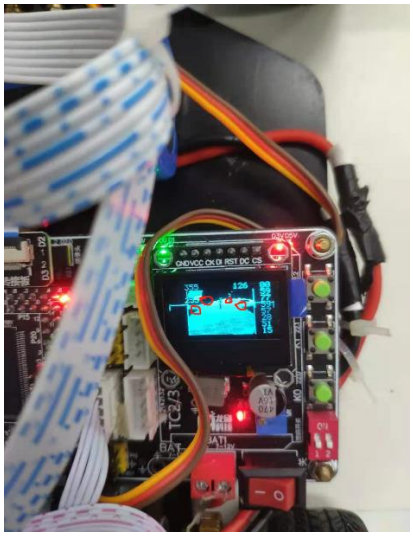
16 届基础四轮技术报告

By：19 级电气 2 班唐某

本届规则中要求使用 B 车模，此车模的转向系统为一个舵机，后轮为单个电机提供动力，并含有机械差速，故在软件层面就不需要考虑两个轮子的差速问题（17 届采用 C 车模就需要考虑）



规则中明确要求了摄像头镜头中心高度离地不得超过 10cm，所以一般将摄像头架设在车头位置，从图片中可以看到摄像头上方还有两根杆子，是为了防止小车冲出赛道撞坏摄像头而做的防护措施，比较粗糙，但不违法规则，可以借鉴一下。若使用摄像头作为主要识别的传感器，则摄像头的角度（即俯角）则非常重要，由于限高的原因，能看到的视野非常有限，所以需要把图像放到显示屏上边看边调摄像头的俯角。建议放在十字路口处，能看到四个拐点为最佳。（个人看法，不喜勿喷）



（如图所示，圈出来的四个点即为所说的拐点，下面还会提到。）

摄像头调整好了就不要轻易动，不然很影响调车，一旦角度变一点，之前设置好的识别的阈值就都不起作用了。所以建议后期直接用热熔胶将摄像头固定死，不要让其有活动的余地。

另外硬件方面的放置也比较讲究，一般在底盘上按照母版的安装孔尺寸自己钻四个孔让

母版尽可能低的安装在底盘上，降低车模重心，提高过弯效率，重心太高了容易在拐弯的时候甩出去，不利于提速。

还有走线问题，在前期尽可能想好怎么走线，最大程度简化，线太长就用扎带捆好找个地方固定住，这样既美观又方便调试，每次上电前检查走线是否准确无误，减小烧板子的概率。特别是电机的信号线，一旦松了一点电机就疯转，建议直接用热熔胶封好。

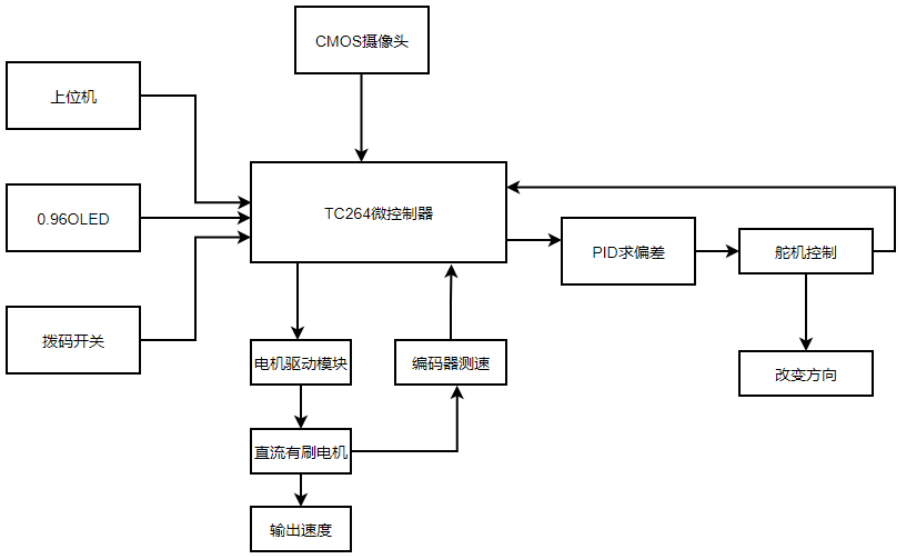
后轮的电机传动轮与轮子的齿轮还有编码器的齿轮要调整至耦合正常，松紧合适，编码器是速度闭环控制的反馈通路，要保证接线无误，齿轮正确咬合，否则很容易使电机疯转。

转向用的舵机如果用的是 sd5，建议去淘宝找店铺:呆萌侠,买他家的金属齿轮的舵机，很耐用，原厂的塑料齿轮舵机基本上一个星期坏一个，又贵又搞心态。

比赛要求自制的电路板如母版驱动板等，我建议是尽早搞定，因为硬件一旦出问题要考虑到物流时间可能会比较长，浪费大量调车的时间，板子提前搞定后面就直接用自己画的板子去调车，尽早舍弃买来的成品板子。

本届基础四轮主控芯片是规则里面要求使用的英飞凌的 TC264DA 单片机，该单片机与 stm32 同样是采用 arm 架构，且资源丰富，容易上手。编译器使用的是 AURIX Development Studio 平台，虽然没有汉化 ，但用习惯了还是比较方便的。每一届使用的芯片都不一样，但核心还是算法，万变不离其宗。

下面是小车运行的程序框图



由于我用的纯摄像头，故单片机只需要处理图像即可。车子由几个部分组成，分别为转向部分（即循迹）、动力部分（车子速度）和调试部分（蓝牙、上位机和显示屏等）。

动力部分：由电机和编码器闭环控制，使用 PI 控制器

```
speed_error=(float)speed_in-Feed_speed;    //计算转速误差
speed_out +=Speed_P*(speed_error-last_error_1)+Speed_I*speed_error+ Speed_D*(speed_error-2*last_error_1+last_error_2);
```

一般用匿名上位机配合蓝牙模块调试，通过发送编码器测得的实际转速的波形来调整 PI 的值。

循迹部分：通过摄像头的每一帧图像，提取使用后的图像经二值化后转化为 0 和 1 存到数组 Pixle[i][j] 中。由下图两个函数实现，函数具体可看代码。

```
Get_Use_Image();
Get_01_Value(0);                // 二值化
```

得到 Pixle 数组后就可以开始找赛道边界了，二值化后的图像只有 黑色和白色，反映到数组里就是 0 和 1，黑色为 0 白色为 1。从图像最底部开始，从中间往左边扫，扫到 1 跳变到 0，此时的横坐标即为左边界；从中间往右边扫，扫到 1 跳变到 0，此时的横坐标即为右边界。经过水平巡线后会得到两个数组 left_point[60]和 right_point[60]，分别代表每一行的左右界的横坐标。每一行的左右边界相加再除 2 就得到这一行的中点，存在中线数组 mid_point[60]中，获得最底部的中线后，往上一行就以最底部的中线的横坐标为起点开始往两边扫，这样保证了边界和中线的连续，也节省了时间。

找到边界后扫线的本质就是遍历数组。在后面图像元素的识别中，其本质也是对数组进行找特征点与不同点

```
Horizontal_line();              //水平扫线，找边界与中线
portrait_line();                //纵向扫线，找特殊元素的拐点
```

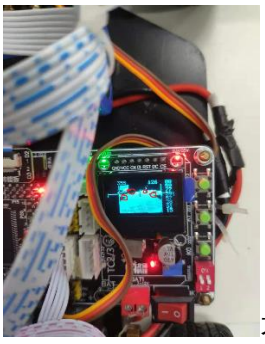
获得中线数组后就可以控制舵机进行转向了，一般采用 20 行到 30 行之间的中线来作为舵机的控制行，车子的速度越快，就需要提前转向，控制的行就要往前移。将中线的横坐标与图像宽度的一半做差（因为摄像头是架设在车子正中间，图像宽度的一半对应的正好是车头正中间的位置），得到的差值即为车子与中线相差的值，反映出来的就是车身偏离赛道中央的距离。得到差值后经过 PD 计算输出到舵机即可循迹了。

赛道元素识别和处理主要有十字，坡道，环岛，三岔和斑马线。

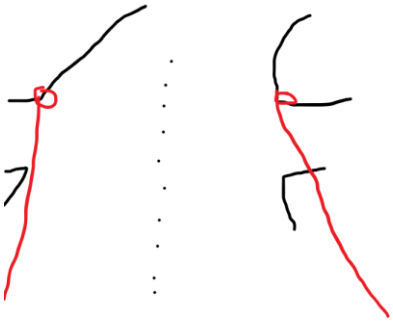
对于智能车，识别元素说简单也简单，说难也难。简单在其本质就是找特征，难在不同元素之间的区分防止误判。

十字路口一般用两边丢线，中间白点数大于某个阈值（实际要放到赛道上找阈值大小），十字的难点在于容易和三岔路误判，还有一种情况是斜入十字，这个比较难，还没遇到，先不讨论。十字标志位置 1 后进入十字处理函数

```
【函数名称】crossroads_dispose
【功　能】十字处理函数
【参　数】无
【返 回 值】无
【实　例】crossroads_dispose();
【注意事项】无
```



这里原本是找四个拐点，不过实际测试后效果并不是很好，拐点容易找错，所以后面改成只找上方两个拐点，然后各自连到左下角和右下角

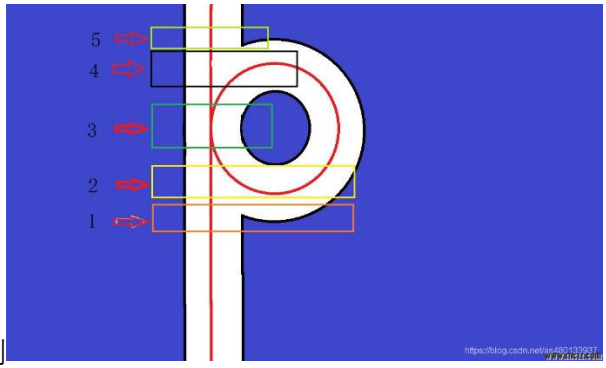


如图所示，经过实际测试发现非常稳定。

坡道我是采用远处赛道宽度突然变宽到一定值，坡道标志位置 1，此时舵机的控制行往下调整几行，等回到正常赛道后解除坡道标志位。

环岛识别和处理，我用了状态机来处理。

这里以右环岛举例



在图片的 1 处之前识别到左边为单调的直线，右边丢线，中间白点的数目与直道相差不大，此时右环岛标志位置 1，右环岛状态位置 1。进入环岛处理函数。走到 2 处会看到两边都不丢线，此时进入状态 2，走到 3 处看到左边不丢线，右边丢线，此时进入状态 3，对入口进行补线处理，找上方的拐点直接往左边拉一条线，实际情况建议拐点的横坐标往左移一段距离，不然会出现内切的现象。进入环岛后，左边第 2 列的白点数会突然减小，此时进入状态 5，走到环岛出口处左边第 2 列的白点数又增大，进入状态 6，开始对出口补线，直接找两个比较合适的点拉一条线即可，等到中间列的白点数和直道相差不大时证明已出环岛回到直道，清空环岛标志位和状态位。



三岔路口，两边丢线，中间列的白点数小于一定值（趴赛道看吧）

找到中间尖尖的拐点往下面直接拉一条线（改图找拐点时找歪了，后面改了阈值，能比较准确的找到中间的点）。比赛时需要两圈里面每圈都走不同条道，我的办法是从发车开始开始计算走过三岔的数量，每走过两次就换方向，实际测试很稳定。这里给出另外一种方法，可以第一圈的时候走左边，过了斑马线后变成走右边。具体可行性自己测试。

斑马线识别，判断黑白跳变点是否大于某个阈值且次数大于某个阈值。判断到斑马线后我是直接把舵机锁定成中值延迟一段时间，否则经常在斑马线前扫线把斑马线当作边界导致中线乱飞。

接下来就是出库入库了，由于时间紧迫，这次出库入库我都是固定打角编码器检测到走一段距离后取消打角，也比较简单。

基础四轮大概处理的东西差不多就这些，还有一些调试的界面不再赘述。个人感觉在没有任何传承的情况下完成所有元素还是比较困难的，尤其在误判上面最容易翻车，需多花时间调整。本文总结了一学期以来做智能车的一些小小的经验，皆在后面的同学不用花大量的时间踩坑，希望有所帮助。